

KARNATAKA RESIDENTIAL EDUCATIONAL INSTITUTIONS SOCIETY

ICT Class 8 - Practical Lab Exam 2

Practical Lab Exam 2 - Answer Key

Class: Class 8

Assessment Type: Practical Lab Exam 2

Total Marks: 20

Duration: 45 minutes

Topics Covered: SQL with Python and Web Development

ICT Teaching Resources Portal

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: SQL Database Operations with Python (5 marks)

Complete Python Program:

- Answer:

```
import sqlite3

# Create database and table
conn = sqlite3.connect('school.db')
cursor = conn.cursor()

cursor.execute('''
    CREATE TABLE IF NOT EXISTS Students (
        roll_no INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        class TEXT,
        section TEXT,
        marks INTEGER
    )
''')

# Insert 8 records
students = [
    (1, 'Alice', '8', 'A', 85),
    (2, 'Bob', '8', 'B', 72),
    (3, 'Charlie', '8', 'A', 90),
    (4, 'Diana', '8', 'C', 78),
    (5, 'Eve', '8', 'B', 88),
    (6, 'Frank', '8', 'A', 65),
    (7, 'Grace', '8', 'C', 92),
    (8, 'Henry', '8', 'B', 70)
]
cursor.executemany('INSERT OR IGNORE INTO Students VALUES (?, ?, ?, ?, ?)', students)
conn.commit()

# 1. SELECT with WHERE clause
print("Students with marks above 80:")
cursor.execute("SELECT * FROM Students WHERE marks > 80")
for row in cursor.fetchall():
    print(row)

# 2. SELECT with ORDER BY
print("\nStudents sorted by marks (descending):")
cursor.execute("SELECT name, marks FROM Students ORDER BY marks DESC")
for row in cursor.fetchall():
    print(f"{row[0]}: {row[1]}")
```

```
# 3. UPDATE operation
cursor.execute("UPDATE Students SET marks = marks + 5 WHERE roll_no = 2")
conn.commit()
print("\nUpdated Bob's marks (+5)")

# 4. DELETE operation
cursor.execute("DELETE FROM Students WHERE roll_no = 6")
conn.commit()
print("Deleted Frank's record")

# 5. COUNT and AVG aggregate queries
print("\nTotal students:", cursor.execute("SELECT COUNT(*) FROM
Students").fetchone()[0])
print("Average marks:", cursor.execute("SELECT AVG(marks) FROM
Students").fetchone()[0])

conn.close()
```

- **Marking:** 1 mark for database and table creation, 1 mark for inserting records, 1 mark for SELECT queries, 1 mark for UPDATE/DELETE, 1 mark for aggregate queries.

Task 2: HTML and CSS Web Page (5 marks)

HTML File (student_portal.html):

- **Answer:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Portal</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <div class="header-content">
      <h1>KREIS Student Portal</h1>
      <nav>
        <a href="#">Home</a>
        <a href="#">Students</a>
        <a href="#">Results</a>
        <a href="#">Contact</a>
      </nav>
    </div>
  </header>

  <main>
    <h2>Student Records</h2>
    <table>
      <thead>
        <tr>
          <th>Name</th>
          <th>Class</th>
          <th>Section</th>
          <th>Marks</th>
          <th>Grade</th>
        </tr>
      </thead>
    </table>
  </main>
</body>
</html>
```

```

        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Alice</td>
            <td>8</td>
            <td>A</td>
            <td>85</td>
            <td>A</td>
        </tr>
        <tr>
            <td>Bob</td>
            <td>8</td>
            <td>B</td>
            <td>72</td>
            <td>B+</td>
        </tr>
        <tr>
            <td>Charlie</td>
            <td>8</td>
            <td>A</td>
            <td>90</td>
            <td>A+</td>
        </tr>
        <tr>
            <td>Diana</td>
            <td>8</td>
            <td>C</td>
            <td>78</td>
            <td>B+</td>
        </tr>
        <tr>
            <td>Eve</td>
            <td>8</td>
            <td>B</td>
            <td>88</td>
            <td>A</td>
        </tr>
        <tr>
            <td>Grace</td>
            <td>8</td>
            <td>C</td>
            <td>92</td>
            <td>A+</td>
        </tr>
    </tbody>
</table>
</main>

<footer>
    <p>&copy; 2024 KREIS Student Portal. All rights reserved.</p>
</footer>
</body>
</html>

```

CSS File (styles.css):

- **Answer:**

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  line-height: 1.6;
}

header {
  background: linear-gradient(135deg, #1a237e, #283593);
  color: white;
  padding: 1rem 2rem;
}

.header-content {
  display: flex;
  justify-content: space-between;
  align-items: center;
  max-width: 1200px;
  margin: 0 auto;
}

nav a {
  color: white;
  text-decoration: none;
  margin-left: 20px;
  padding: 8px 16px;
  border-radius: 4px;
  transition: background 0.3s;
}

nav a:hover {
  background: rgba(255, 255, 255, 0.2);
}

main {
  max-width: 1200px;
  margin: 2rem auto;
  padding: 0 2rem;
}

h2 {
  margin-bottom: 1rem;
  color: #1a237e;
}

table {
  width: 100%;
  border-collapse: collapse;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

th, td {
```

```

padding: 12px 15px;
text-align: left;
border-bottom: 1px solid #ddd;
}

th {
  background-color: #1a237e;
  color: white;
}

tbody tr:nth-child(even) {
  background-color: #f5f5f5;
}

tbody tr:hover {
  background-color: #e3f2fd;
  transition: background 0.3s;
}

footer {
  background: #333;
  color: white;
  text-align: center;
  padding: 1rem;
  margin-top: 2rem;
}

@media (max-width: 768px) {
  .header-content {
    flex-direction: column;
    text-align: center;
  }

  nav {
    margin-top: 1rem;
  }

  nav a {
    margin: 0 10px;
  }

  table {
    font-size: 14px;
  }

  th, td {
    padding: 8px 10px;
  }
}

```

- **Marking:** 1 mark for HTML structure, 1 mark for table with data, 1 mark for CSS styling, 1 mark for alternating rows and hover, 1 mark for responsive design.

Task 3: SQL Query Challenge (5 marks)

Complete Python Program:

- **Answer:**

```

import sqlite3
from datetime import datetime

conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# Create tables with Foreign Key
cursor.execute('''
    CREATE TABLE Employees (
        emp_id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        department TEXT,
        salary REAL,
        joining_date DATE
    )
''')

cursor.execute('''
    CREATE TABLE Projects (
        project_id INTEGER PRIMARY KEY,
        project_name TEXT,
        emp_id INTEGER,
        start_date DATE,
        end_date DATE,
        FOREIGN KEY (emp_id) REFERENCES Employees(emp_id)
    )
''')

# Insert employees
employees = [
    (1, 'Alice', 'IT', 75000, '2021-03-15'),
    (2, 'Bob', 'HR', 65000, '2020-07-20'),
    (3, 'Charlie', 'IT', 80000, '2019-01-10'),
    (4, 'Diana', 'Finance', 70000, '2022-05-25'),
    (5, 'Eve', 'HR', 68000, '2021-11-30'),
    (6, 'Frank', 'IT', 72000, '2020-09-05')
]
cursor.executemany('INSERT INTO Employees VALUES (?, ?, ?, ?, ?)', employees)

# Insert projects
projects = [
    (1, 'Website Redesign', 1, '2023-01-01', '2023-06-30'),
    (2, 'Mobile App', 1, '2023-03-15', None),
    (3, 'HR Portal', 2, '2023-02-01', '2023-08-31'),
    (4, 'Data Analytics', 3, '2023-04-01', None),
    (5, 'Finance System', 4, '2023-05-15', '2023-12-31'),
    (6, 'Employee Training', 2, '2023-06-01', None),
    (7, 'Security Audit', 3, '2023-07-01', None),
    (8, 'Budget Planning', 5, '2023-08-01', '2024-01-31'),
    (9, 'IT Support', 6, '2023-09-01', None),
    (10, 'Cloud Migration', 1, '2023-10-01', None)
]
cursor.executemany('INSERT INTO Projects VALUES (?, ?, ?, ?, ?)', projects)
conn.commit()

# 1. Employees earning more than average salary

```

```

print("\n1. Employees earning more than average salary:")
cursor.execute('''
    SELECT name, salary FROM Employees
    WHERE salary > (SELECT AVG(salary) FROM Employees)
''')
for row in cursor.fetchall():
    print(f"    {row[0]}: ₹{row[1]}")

# 2. Departments with more than 2 employees
print("\n2. Departments with more than 2 employees:")
cursor.execute('''
    SELECT department, COUNT(*) as emp_count
    FROM Employees
    GROUP BY department
    HAVING COUNT(*) > 2
''')
for row in cursor.fetchall():
    print(f"    {row[0]}: {row[1]} employees")

# 3. Employees working on most projects
print("\n3. Employees working on most projects:")
cursor.execute('''
    SELECT e.name, COUNT(p.project_id) as project_count
    FROM Employees e
    JOIN Projects p ON e.emp_id = p.emp_id
    GROUP BY e.emp_id
    ORDER BY project_count DESC
    LIMIT 1
''')
for row in cursor.fetchall():
    print(f"    {row[0]}: {row[1]} projects")

# 4. Total salary expenditure per department
print("\n4. Total salary expenditure per department:")
cursor.execute('''
    SELECT department, SUM(salary) as total_salary
    FROM Employees
    GROUP BY department
    ORDER BY total_salary DESC
''')
for row in cursor.fetchall():
    print(f"    {row[0]}: ₹{row[1]}")

# 5. Ongoing projects (no end date or end date > today)
print("\n5. Ongoing projects:")
today = datetime.now().strftime('%Y-%m-%d')
cursor.execute('''
    SELECT p.project_name, e.name as emp_name
    FROM Projects p
    JOIN Employees e ON p.emp_id = e.emp_id
    WHERE p.end_date IS NULL OR p.end_date > ?
''', (today,))
for row in cursor.fetchall():
    print(f"    {row[0]} (Assigned to: {row[1]})")

conn.close()

```


- **Marking:** 1 mark for table creation with foreign key, 1 mark for data insertion, 1 mark for subquery and JOIN queries, 1 mark for aggregate queries, 1 mark for ongoing projects query.

Viva Voce (5 marks)

1. What is the difference between INNER JOIN and LEFT JOIN?

- **Answer:** INNER JOIN returns only matching records from both tables. LEFT JOIN returns all records from left table and matching records from right table (NULL for non-matching).
Example: `SELECT * FROM A INNER JOIN B ON A.id = B.a_id` vs `SELECT * FROM A LEFT JOIN B ON A.id = B.a_id`.
- **Marking:** 1 mark for explaining both joins with examples.

2. How does CSS specificity work?

- **Answer:** Specificity determines which CSS rule applies when multiple rules target same element. Priority: Inline styles (highest) > Internal/External styles > Browser default (lowest). Within same level, ID selectors > class selectors > element selectors.
- **Marking:** 1 mark for explaining specificity levels.

3. What is the purpose of a Foreign Key in SQL?

- **Answer:** Foreign Key creates a link between two tables by referencing the primary key of another table. It maintains referential integrity by preventing invalid data entries and enabling relationships between related data.
- **Marking:** 1 mark for explaining purpose and data integrity.

4. What is the difference between WHERE and HAVING in SQL?

- **Answer:** WHERE filters rows before grouping (cannot use aggregate functions). HAVING filters groups after GROUP BY (can use aggregate functions). Example: `WHERE salary > 50000` vs `HAVING AVG(salary) > 40000`.
- **Marking:** 1 mark for explaining both clauses with examples.

5. Explain difference between responsive and adaptive design.

- **Answer:** Responsive design uses fluid grids and media queries to adapt to any screen size continuously. Adaptive design uses fixed layouts for specific screen sizes and switches between them. Responsive is more flexible; adaptive can be more performant.
- **Marking:** 1 mark for explaining both approaches.